CG2271 Real-Time Operating System
GitHub/ehz0ah
NUS CEG Lee Hao Zhe

Processes

• A process is **an instance** of a running program and **MUST** have CPU time allocation (virtual CPU, the role of OS), Memory (real or virtual), Process ID and Threads. Process will not execute if any of the requirements are **not fulfilled**.

• Each process operates under the illusion that it has all to execute. Processes communicate and synchronize with each other through Inter-Process Communication (IPC).

Memory Layout of a Program/Process

Code or Text Segment: Contains binary instructions to be executed. Read-only clone of the program. OS knows the space needed and memory's contents. Program Counter (PC): Point to the next instruction.
Static Data Segment: Includes initialised global, constant, and static local variables. Shared between threads.
BSS Segment: Uninitialized global/static variables are set to zero.
Heap: Dynamic memory allocation (malloc/free). Used for allocating memory during program execution. OS knows nothing in advance.
Stack: Used for procedure calls and returns. Stack Pointer (SP): Manages the last-in, first-out (LIFO) order of operations.

Process Abstraction

Switching between process means the OS must keep track of **all the execution context** including Memory State (code, data, heap and stack), CPU state (PC, SP and other registers), as well as the OS state.
Thread: A lightweight process; A process may have multiple threads which share the same system resources.

Process Control Block (PCB) / Task Control Block (TCB)

• PCB maintains all the relevant information for the process including Process ID, Process state, Registers, Scheduling information (priority) and Memory management information.

• The Process ID points to the entry in the **process table** where the pointer to the PCB is stored.

Process Creation

Created by initialization, or by request of a process. Assigns unique ID for the process and allocates memory for the PCB and other control structures (kernel) and user memory. Initialize the PCB and memory management.

Process Termination

Usually stopped by the OS/User or terminate itself. OS handles the output of the process, releasing the resources, reclaim the memory and unlink the PCB. In embedded software, some processes may never terminate as terminating usually implies a fault.

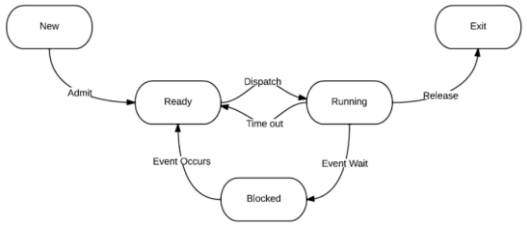
Context Switching

PCB makes context switching a bit easier. Scheduler will start or stop a process accordingly. When switching from process A to B, OS **stores** necessary information (Hardware registers, Program Counter, Memory states, stack and heap, State) in PCB A. Similarly, OS **loads** necessary information from the PCB B to run process B. Context switching **does consume time**. It could take up to several thousand CPU cycles. Overhead and bottleneck requires hardware support.

Process States

Many variants but a standard model has **FIVE states**:
New: Process just been created, not ready for the queue.
Ready: Process can be loaded by the OS.
Running: Scheduler picked the process from the **queue** based on priority and executed it, usually **only one at a time**. (Single core)
Blocked: Process not using CPU resources and not in the queue, usually waiting for some resources.
Exit: Process finished, needs to terminate.

State Transition



Admit: Process is fully loaded into memory and control is established.
Dispatch: Scheduler assigns CPU to the process.
Time out: Expired or **pre-empted**, pushed back to the queue.
Event Wait/Event Occurs: Generally, requests that **cannot** be met now, need wait until something occurs (Resources, input, etc.).
Release: Release resources and end the process.
Context switch takes place whenever a process leaves/enters the running state.
Round Robin: When tasks have equal priority, each task will be given a fixed CPU time to run **(Time Slice). Pre-emption can still occur.**

Multi-Processing

SISD: Sequential without parallelism. Concurrency at hardware level.
SIMD: A single program on many processing units e.g. Graphics Processing Units (GPUs), vector processors.
Multiple Instructions Single Data (MISD): Rarely used
MIMD: Many processors executing different instructions on different data (Distributed systems, Shared memory).

Race Condition

• When a set of processes **access shared resources** to carry out a computation and the results of the computation depend on the **exact** way the processes **interleave**. Race conditions undermine the **correctness** of any concurrent system and should be **eliminated**.

• Taking a correct piece of sequential code (Non-Atomic) and using it for concurrent programming may not work as expected.

• Results are incorrect **only sometime**. Value and correctness of result depend on the **precise timing relationship** between the processes. Completely **non-deterministic**. Hard to find and fix bugs.

Critical Section

Regions of code accessing shared/mutually exclusive resources are called **critical regions** or **critical section.**

Mutual Exclusion (Mutex)

Mutex **controls access** to shared resources, enforced to ensure **only one** thread of execution can have access at any time.

Requirements of a Good Mutex:

1. The only reason a request to a critical section is rejected/delayed is that another process is accessing already.
2. Process can only access the critical section for a **finite** time.
3. No **Deadlock** and **Starvation.**

```
osMutexId_t myMutex;      // Declared globally
myMutex = osMutexNew(NULL) // Create Mutex, Default Attribute
osMutexAcquire(myMutex, osWaitForever) // Acquire Mutex
osMutexRelease(myMutex) // Release Mutex
```

Semaphores

A semaphore is a container of **several** tokens. Required to acquire a token first before accessing a resource. When finished with the resource, return the token. Also used to **synchronize** tasks or protect variables and resources. Has a **priority queue** to put calling process that are blocked.

```
osSemaphoreId_t mySem;    // Declared Globally
mySem = osSemaphoreNew(1,1,NULL) // Max Token = Initial = 1
osSemaphoreAcquire(mySem, osWaitForever) // Acquire token
osSemaphoreRelease(mySem) // Release token
```

Mutex vs Semaphore

Semaphore: Used for signaling from tasks or ISRs to waiting tasks. Can be initialized to 0, meaning "The event hasn't happened yet".
Mutex: Used to ensure mutually exclusive access to a shared object. It's a **binary semaphore** with **priority inheritance**. Always initialized to 1, meaning "The object isn't being used now".

Deadlock (No Perfect Solutions Yet)

All processes are waiting for others to finish so that required resources will be released, while reluctant to give up the resources that are also required by others. Possible Sol: Acquire resources using the same order
Deadlock Conditions: (Necessary but **NOT** sufficient)
1. **Mutual exclusion:** Only one process can use a resource at a time.
2. **Holding:** Process requesting additional resources while holding one.
3. **No Pre-emption:** Resource can be released only voluntarily by the process holding it after the process has completed its task.
4. **Circular Wait:** There exist a set $\{P_0, P_1, \ldots, P_N\}$ of waiting processes such that P_0 is waiting for P_1, and P_1 is waiting for P_2, etc.

Scheduling

A set of rules (policy) for **allocating tasks** to the processor(s). Goal is to ensure **ALL** tasks meet their respective deadlines.

Basic Real-Time Task Model

Worst-Case Execution Time (WCET) (C): Maximum amount of time required to complete the execution of any instance of task without any interference from other activities.
Worst-Case Response Time (WCRT) (R): Maximum elapsed time between the release of a task instance and its completion. (Pre-emption)
Deadline (D): Maximum amount of time **allowed** between a task being released and its completion time.

Assumptions:

1. Concurrent Program consists of fixed number **(N) of tasks**.
2. Tasks are periodic with known **Periods (P)**. Period do not change with time. A task is an infinite sequence of **instances**. One task instance is **released** (becomes ready) at the beginning of each period.
3. Tasks are completely **independent** of each other.
4. Negligible system overhead for scheduling and context switch time.
5. Deadline of each task equal to period. D = P

Clearly, C ≤ D ≤ P & C ≤ R. To **meet deadline**, we need R ≤ D.

Pre-emptive Priority-Driven Scheduling

The policy in which a lower-priority task that is currently executing gets context switched out once a higher-priority task become ready.

Rate Monotonic Scheduling (RMS)

• **Fixed-priority** assignments in periodic tasks

• Priority is **inversely proportional** to task period. Shorter the task period, higher the priority. RMS does not depend on WCET.

RMS: Optimal Fixed-Priority Scheduling

• If a task set is schedulable by any arbitrary but fixed-priority assignment, then it is schedulable by RMS as well.

• Number of possible priority assignments is equal to the number of permutations of N tasks = N!.

• No need to exhaustively try out all the priority assignments because RMS is guaranteed to pick the right one.

Hyper Period (HP)

• The period of one unit of the behaviour before it repeats. Least Common Multiple (LCM) of the task periods.

• If you simulate the behaviour of the schedule over one hyper-period and no deadline is missed, then the task set is **schedulable**.

Schedulability Analysis

• Analysing the system and predicting its **worst-case behaviour** with respect to timings when a scheduling algorithm is applied.

• Validating that the worst-case behaviour **meets** the timing constraints imposed on the system at design time.

Utilization of a periodic task set is the fraction of processor time spent in the execution of the task set.
CPU Utilization (U) = Add up WCET/Period for all task in the task set.

• U ≤ 1 is a **necessary** condition for any task set to have a feasible schedule but **NOT SUFFICIENT** to have one **under fixed priority**.

• Use the **optimality of RMS** to check the schedulability of a task set under fixed priority. Step 1: Assign priorities according to RMS. Step 2: Simulate the schedule for hyper-period. Step 3: If no deadline is missed during hyper-period, task set is schedulable. (Expensive for large HP)

• **Sufficient Condition (Utilization Bound):** A task set with n tasks is schedulable with **fixed priorities** if $U \leq n\,(2^{1/n} - 1)$.

• A task set may still be schedulable even if the utilization bound is not satisfied. **(NOT A NECESSARY CONDITION)**

Critical Instant (All tasks ready to execute simultaneously)

• The instant at which the release of the task will produce the largest **response** time. (Lower priority task will keep getting pre-empted)

• **Theorem:** A critical instance for any task occurs whenever it is release simultaneously with the release of **ALL higher-priority tasks.**

Critical Instance Analysis (RMS)

• WCRT of a task T at its critical instance must be less than its deadline.
Imagine a case where the addition of T3 to T1 & T2 cause U > Utilization Bound for 3 tasks. (Priority of T1 & T2 > Priority of T3)
Step 1: $S_{i,0} = \sum_{j=1}^{i} C_j$, i in this case is 3. (Use $S_{i,0}$ in the next step)
Step 2: $S_{i,x+1} = C_i + \sum_{j=1}^{i-1} \left(C_j \times \left\lceil \frac{S_{i,x}}{P_j} \right\rceil \right)$, i in this case is 3.
Step 3: Repeat Step 2 until $S_{i,x+1} = S_{i,x}$. If S_{Final} < T3's Deadline/Period, the task set T1, T2, T3 is schedulable.

• The point of Step 1 is to sum up the WCET of all tasks that are higher priority than T3 and T3. Sort the task set if needed.

Earliest Deadline First (EDF) Scheduling

• **Dynamic priority** scheduling scheme where task with the **earliest deadline has the highest priority.**

• Task Priority depends on the **current deadline** of the **active** task instances. Task priority may be updated **only** when any new task instance is released. When two tasks have the same deadline, the one running continues running to reduce context switching overhead.

• Sorting is called to get the earliest deadline task. **Computationally intensive** for embedded systems (Weaker CPU) → Not popular in the real world compared to fixed priority scheduling.

• EDF is an **optimal** scheduling policy. EDF can **always** produce a feasible schedule if U ≤ 1. Scheduling with dynamic priority is feasible **if and only if** U ≤ 1 **(NECESSARY & SUFFICIENT).**

Co-operative Multi-Tasking Scheduling

All tasks have the same priority and Round-Robin is disabled. Each task will run until it reached a blocking OS call or voluntarily yields the CPU.

Real-Time Operating Systems (RTOS)

RTOS are designed to meet **harsh timing constraints**. It is predictable, deterministic, fast, responsive, fail-safety and has advanced scheduling.
Huge Sequential Loop: Lengthy ISR. Need to synchronise between ISRs. Poor Predictability (Nested ISRs) & Extensibility. Any change result in ripple effect throughout the entire system.
RTOS: All computation requests are encapsulated into tasks and scheduled on demand. Multi-Tasking. Concise ISR (Deterministic).
Priority Inversion is a scenario in RTOS where a higher-priority task is indirectly pre-empted by a lower-priority task due to resource sharing.
Priority Inheritance is elevating a low priority task to the priority of the higher priority task that is blocked by it. It resumes its original priority after finishing using the resource. Only works for Mutex.
Priority Ceiling is predetermined by the programmer as the highest priority among **all threads that might** access the resource. Any task that acquired the shared resource is raised to the priority ceiling.

Flags

Thread Flags are not created as they exist automatically within each thread. They are 32-bit word with 31 Thread Flags available.

```
osThreadId_t tid;       // Declare Globally, tid = Thread ID
tid = osThreadNew(threadX, NULL, NULL); // Create Thread
osThreadFlagsSet(tid, 0x0001);  // Set bit 0
osThreadFlagsWait(0x0001, osFlagsWaitAny, osWaitForever);
osFlagsWaitAny = Wait for any flag (Default).
osFlagsWaitAll = Wait for all flags.
osFlagsNoClear = Specified flags are not cleared.
```

Events

Unlike Flags, Events are not tied to any Thread. Need to explicitly create a new Events Flag object. Each object will have 31 Event Flags.

```
osEventFlagsId_t eid;  // Declare Globally, eid = Event ID
eid = osEventFlagsNew(NULL);  // Create Event Flag
osEventFlagsSet(eid, 0x0001); // Set bit 0
osEventFlagsWait(eid, 0x0001, osFlagsWaitAny, osWaitForever);
```

Message Queue (FIFO → Sort Based on Priority)

Message passing is another basic communication model between threads. One thread sends data, while another thread receives it. It's more like I/O rather than direct access to shared information.

```
osMessageQueueId_t mid;  // Declare Globally, mid = Message ID
mid = osMessageQueueNew(1, sizeof(dataPkt), NULL);
osMessageQueuePut(mid, &myData, NULL, 0);
osMessageQueueGet(mid, &myRxData, NULL, osWaitForever);
```

Message Queue allows threads to communicate and synchronise with each other while transferring large amounts of data at the same time.

Re-entrant Function (Inter-Thread/ISR Safe)

It is a function that can be safely executed by multiple threads concurrently without causing any undefined behaviour, shared data corruption & race conditions. It avoids using/modifying static/global variables & every entry/call to the function is a completely new instance.

Memory

ARM is a Load/Store Architecture – Variables are only modified in registers, not in memory. Any memory-resident variable must be accessed with at least three instructions: load, modify, store. Not atomic.

Memory Leaks refers to "losing" references of dynamic memory.

Dangling Pointers point to data that's not there anymore.

Logical Memory (Virtual Memory) refers to the memory addresses used by a program. It provides an abstraction layer that gives each process the illusion of having its own contiguous block of memory.

Physical Memory, on the other hand, refers to the DRAM in your computer. This is where data is physically stored.

Memory Management Unit (MMU) creates an illusion by translating the virtual addresses used by programs into physical addresses.

Internal fragmentation occurs when a block of memory allocated to a process is larger than the process needs, leaving unused memory within the allocated block. Fixed size allocation may lead to internal fragmentation, but less overhead to keep track of memory.

External fragmentation occurs when free memory is available but is fragmented into small, non-contiguous blocks that cannot be allocated to a process needing a large contiguous block. Can be compacted.

Free List is a data structure used to keep track of free memory blocks available for allocation. It typically consists of a doubly linked list of free memory blocks (Hole). Each node in the list represents a hole, with pointers to the next hole. It reduces overhead of searching free memory. Place deallocated memory into free list and merge with hole before/after if possible.

Memory Allocation Algorithms

Best Fit: Pick smallest hole that will satisfy the request. Must search entire list every time. Tends to leave lots of small holes. External Frag.

Worst Fit: Pick the largest hole to satisfy the request. Must search entire list every time. Still leads to fragmentation issues.

First Fit: Pick the **first** hole large enough to satisfy the request.

Next-Fit: Start search from where first fit left off. Much faster than best or worst fit. Still has fragmentation issues like best fit. (Popular)

1. First Fit always starts the search from the beginning. Over time, the holes close to the beginning will become too small, so the algorithm will have to look further down the list, increasing the search time.
2. Next Fit avoids the above problem by changing the starting point of the search. This in turn leads to a more uniform distribution of hole sizes across the free list and will therefore lead to faster allocation.

Compaction

It addresses external fragmentation by compacting holes into a single contiguous block of free memory. It moves all chunks to one end, leaving a large hole on the other end. It can be done on the heap level but expensive/difficult to do it on a process level.

Multiple Free Lists/Buddy System O(log n)

Looks like an adjacency list where memory is divided into blocks of different 2^n sizes. Each block size has its own free list that keeps track of available memory chunks of that specific size. When allocating, break current hole into two equal size holes and allocate one of them if it roughly matches the size of the request. When freeing, if the chunk returned **and** its buddy are in the free list, merge them.

Virtual Memory (VM)

Both SRAM (0.5-5ns) & DRAM (50-70ns) are fast. SRAM > DRAM. Magnetic Disk (Hard Disk): Slow, Average latency is about 5-20ms.

Idea: Keep small but fast memory near CPU & large but slow memory farther away from CPU. Registers > SRAM > DRAM > Hard Disk.

Temporal Locality: If a variable is accessed once in a program, it is likely to be accessed again soon.

Spatial Locality: If an item is referenced, nearby items will tend to be referenced soon.

A **working set** refers to the set of virtual pages that a process actively uses during a specific period. Put frequently used data into the working set which is close to the CPU to prevent access to slower storage.

Cache is a small but fast SRAM near CPU and is hardware managed while virtual memory is OS managed. (Both transparent to programmer)

Virtual Memory is achieved by making a portion of the **hard disk** look like memory to the CPU. Each process has its own private virtual address space. VM can be seen as an array of $N = 2^n$ contiguous bytes stored on a disk that is partitioned into number of blocks called page of size $P = 2^p$ bytes. Physical Memory = $M = 2^m$, where N > M.

Virtual Memory Page Status

Unallocated: The page is not being used by the process.

Uncached: Page is used by the process, but it is not cached in DRAM.

Cached: Page is being used by the process and it is cached in DRAM

Page Offset is a part of a virtual or physical address that identifies the exact location within a page. Requires k bits such that 2^k = Page Size. Number of Virtual/Physical Page = Size of VA/PA ÷ Page Size.

NOTE: Offset NOT stored in the MMU! Transfer VP → PP directly!

Indirection refers to the concept of using a level of abstraction (Mapping/Page Table) to access physical memory addresses.

A **page table** is an array of page table entries (PTE) that maps virtual pages (VP) to physical pages (PP). One page table/process. **Stored in DRAM**. One VP corresponds to one PTE. Each PTE have one valid bit.

Page Hit is when the requested VP is cached in DRAM. Valid bit = 1.

Page Fault is when the requested VP is still in disk. Valid bit = 0.

Page Table Size in bytes = No. of VP * **ceil**(Bits in each PTE ÷ 8).

Miss Penalty: An exception (interrupt) is raised during page fault, the OS page handler loads page from disk to DRAM, modify PTE's valid bit of that VP to 1 and return to faulting instructions (Restart).

Page Replacement Algorithm: If DRAM is full, one of the cached VP is evicted to have space for new VP (Swap). But before swapping, the OS checks the **dirty bit** to determine whether the cached VP has been edited. If the cached VP was edited (Dirty Bit = 1), the cached VP is saved into hard disk first before its replaced with the new VP.

Memory Protection & Sharing: Multiple VP can map to one PP. A process cannot access PP it does not have mapping for. Extend PTE with additional permission bits (Read, Write, Execute, etc) which are checked on every memory access. If violated, raise exception.

Translation Lookaside Buffer (TLB) is a small, hardware cache in the MMU, between SRAM/Cache & DRAM/Memory, that works the same as a page table but smaller number of VPs. Due to locality, a TLB with 64/128/256 entries can attain close to 99% hit rate as majority of accesses will stay within a few pages. UNLIKE page table where only PPN + Valid Bit is stored, TLB also stores VPN.

Sequence for memory access: CPU generates virtual address for the MMU. The MMU breaks down the virtual address to retrieve the VPN and checks the TLB. If it's a **TLB hit**, PPN is returned to the MMU and then appends to the offset to form the physical address to access DRAM. In the scenario that it's a **TLB Miss**, the MMU will use the virtual address to generate the PTE address to query the table in DRAM to get the PPN and store it back to TLB. PPN is then returned to the MMU and then appends to the offset to form the physical address to access DRAM.

Direct Memory Access (DMA)

DMA moves data between a peripheral and the CPU's memory without the direct intervention of the CPU itself. Provides the fastest possible means of transferring data between an interface and memory. It requires no CPU overhead and leaves the CPU free to do useful work.

DMA Controller (DMAC) is a dedicated hardware component that takes control of the **data and address buses** to perform DMA operations. It is not necessarily initiated by the Peripheral as it can be initiated by the processor itself. Many DMACs can handle several interfaces, which requires their registers to be duplicated. Each interface is referred to as a channel. **Three modes of DMA operations**:

Burst Mode: DMAC seizes system bus for the duration of data transfer. Allows data to be moved at the fastest possible rate within the memory/bus/interface timing requirements. CPU is effectively halted in the burst mode because it cannot use its buses.

Cycle Steal Mode: DMA operations are interleaved with the processor's normal memory accesses. DMAC requests control of the buses, temporarily halts the CPU's memory access, performs a small data transfer, and then returns control to the CPU. This process repeats until all data is transferred. The resulting delay is minimal and negligible for most CPU tasks, ensuring efficient operation.

Transparent Mode: DMA operations are **only done** when the CPU is idle. DMAC monitors the system bus and identifies idle cycles and gains access to the bus only when it detects an opportunity to transfer data without impacting the CPU. The detection relies on hardware logic that observes signals like memory read/write or peripheral activity. Due to the **need for monitoring** CPU & bus activity, design is more complex and overall transfer speed may be slower compared to other modes.

Additional Information

1. Fixed-Size Partition's overhead is constant as # of partitions are fixed.
2. Dynamic-Size Bitmap Partition's overhead is constant as size of bitmap is determined by the number of blocks into which the memory is divided, regardless of how many partitions have resulted from allocation at any given moment. (Point 26 for definition of Bitmap)
3. EDF better utilise the processor than RMS as for $U \leq 1$, EDF is guaranteed to produce a feasible schedule while RMS can still fail.
4. Page fault can still happen even if the memory usage of the task is smaller than the DRAM capacity as OS usually uses demand paging to cache the data from disk to DRAM. (Cached after first memory access)
5. **Unbounded priority Inversion** is when the high priority task is blocked by the low priority task in which is pre-empted by a medium priority task and so on. Now, high priority task is the last to execute.
6. Mutex have the idea of **ownership**. Only the thread that acquired the mutex is allowed to release it. This is the reason why priority inheritance only works for mutexes and not semaphores.
7. Necessary condition means that the condition must be satisfied for a feasible schedule to even exist. Sufficient condition means that if the condition is satisfied, then a feasible schedule will exist.
8. Utilization Bound (U) for **infinite tasks** is $\ln(2) \approx 0.693$. From zero to U, task set is 100% schedulable. From U to one, task set may or may not be schedulable. $U \geq 1$ is 100% not schedulable.
9. Existence of a fixed priority schedule implies an existence of an RMS schedule. Non-existence of an RMS schedule also implies the non-existence of a fixed priority schedule. Use critical instance analysis.
10. When doing critical instance analysis (CIA), start the calculation of CPU utilisation with the highest priority task and check if its below utilisation bound. If it is, continue adding the next highest priority task until you find an addition that exceeds utilisation bound.
11. Virtual pages are also known as **"pages"** while physical pages are known as **"frames"**.
12. **1 KiB = 1024 Bytes, 1KB = 1000 Bytes**.
13. **ALL** addresses generated by the CPU data path or pipelines are always virtual addresses. The primary place of production is in the Program Counter (PC). **Fetch stage**: Generate virtual addresses of program instructions to be fetched from memory. **Memory stage**: Generate virtual addresses for reading/writing data from/to memory.
14. **Kernel Mode**: Full access to system resources. Used by the OS to perform privileged operations.
15. **User Mode**: Restricted access to system resources. Used by applications to ensure stability and security.
16. The **Banker's Algorithm** dynamically checks resource allocation to ensure the system remains in a safe state. It prevents deadlocks by only granting resource requests if the system can still satisfy all tasks' maximum resource demands.
17. **Watchdog Timer** is a hardware timer that resets the system if the software fails to reset it periodically. Used to recover from software hangs or crashes.
18. **Starvation** occurs when a low-priority task is perpetually blocked because higher-priority tasks monopolise resources. **Priority aging** can be used to mitigate starvation.
19. Keep ISRs short and efficient to minimize interrupt latency. Avoid blocking calls or long loops inside ISRs. Use flags or queues to defer complex processing to tasks.
20. **Mailbox** sends fixed-size messages between tasks while message queue sends variable-sized messages.
21. **Atomic operations** are usually 1-step and are completed without interruption, ensuring data consistency in concurrent environments.
22. **Idle task** is a task run by the CPU when there is no other task to run.
23. Mutex **cannot be released** in an ISR.
24. The **volatile keyword** tells the compiler that the value of a variable may change at any time, without any action being taken by the code the compiler finds nearby. This prevents the compiler from optimizing the code in a way that assumes the value of the variable cannot change unexpectedly. It ensures that the compiler reads the actual value from the hardware register each time it is accessed.
25. **MISRA C** is a set of software development guidelines for the C language, specifically designed to facilitate code safety, security, portability, and reliability, particularly in the context of embedded systems programmed in compliance with ISO C (C90/C99) standards.
26. **Bitmap**: Maintains a 1-bit array in the OS with each bit corresponds to the state of 1 block (allocation unit determined by OS)